INTEGRATION PROJECT — SOLO

Pitwall

A platform connecting every part of a karting track.

7 services | Message bus | Self-hosted | Free

What is Pitwall?

Pitwall is a platform for karting tracks that connects all operational systems into one integrated whole. A karting track runs many things at once: drivers show up and get registered, sessions start and laps get recorded, the bar processes orders, the back office sends invoices, and the TV screens show a live leaderboard. Right now all of these are separate, disconnected pieces. Pitwall connects them.

The key architectural principle is that none of these services talk to each other directly. They all publish and listen to a shared message bus in the middle. If one service goes down, the rest keep running independently.

Loosely coupled
Every service is independent. If billing goes down, timing still works.

Event-driven
Every meaningful action produces an event. Other services react to those events.

Fault tolerant
No single service going offline should block another. Degrade gracefully.

No dead ends
Every edge case must have a handled outcome. Always a path forward.

The services

T

Timing

Scans drivers in, records laps, calculates times and personal records. Includes a simulator for dev.

F

Frontend

The public website where drivers register, book sessions, and view their lap history.

B

Booking

Manages the session schedule: heats, available spots, cascading changes when sessions run late.

D

Driver

Stores driver profiles and full lap history across all sessions. Source of truth for driver identity.

$

Billing

Handles session charges and bar orders. Invoices for companies, receipts for individuals.

M

Mailing

Sends automated emails triggered by events: confirmations, summaries, personal bests, invoices.

L

Leaderboard

Live standings on track screens during an active session. Updates in real time.

What every service must deliver

Regardless of what a service does, every one is responsible for the same baseline:

- Runs in a Docker container
- Publishes relevant events to the message bus
- Listens and reacts to relevant events from other services
- Validates all incoming messages — logs errors on bad data
- Exposes a /health endpoint
- Sends a heartbeat signal every second
- All code lives in Git (main, dev, prod branches minimum)
- Automated deployment pipeline on push

Loosely coupled. If billing goes down, drivers can still scan in, laps still get timed, and the leaderboard still updates. No service should be blocked because another is unavailable. This is the most important architectural rule in the project.

Monitoring

A separate control room watches every service. It does not sit on the message bus — instead it checks each service's health endpoint directly, so it can detect a service that has gone completely silent. When something goes down, an alert fires immediately.

The control room provides a dashboard with live status (online / offline) for every service, running statistics (active drivers, sessions today, laps recorded), and a full alert history.

Edge cases to handle

The system must handle the following without breaking. Think through the sad paths for each service — the answer should never be an error with no way forward.

- A session runs late, requiring downstream sessions in the schedule to shift automatically

- A private driver (not linked to a company) requests a formal invoice after a session
- A service restarts mid-session and needs to catch up on missed events
- Conflicting driver data arrives from two services at the same time
- The scanner goes offline mid-session — laps recorded before the outage must not be lost
- A driver tries to book a session that is already full

Suggested build order

1	**Message bus + control room** Nothing else can be validated without these running first.
2	**Timing** First end-to-end slice — start with the simulator, add real hardware later.
3	**Leaderboard** Now you can see data flowing visually in real time.
4	**Driver** Driver identity becomes real across sessions.
5	**Booking** Sessions have structure and a schedule.
6	**Frontend** External users can register and book sessions.
7	**Billing** The financial layer is live.
8	**Mailing** Communication loop is closed.
9	**Harden** Validation, error logging, sad paths, deploy pipelines across all.